

NumPy, short for *Numerical Python*, is a foundational Python library built specifically for heavy numerical computing and data processing.

Why Python Needs NumPy

- **The Problem:** Standard Python lists store objects dynamically, causing massive overhead when managing millions of numbers. Additionally, multiplying a normal Python list by an integer repeats its elements (`[1, 2] * 2` becomes `[1, 2, 1, 2]`) rather than applying math to each element.
- **The Solution:** NumPy is written natively in **C**, making it magnitudes faster. It introduces the **NumPy Array**, which enables vectorized operations (applying a mathematical calculation to an entire dataset simultaneously without explicit loops).

Installation & Initialization

Open your terminal (**Alt + F12**) and run:

```
Bash  
pip install numpy
```

In your Python script, import the library using the standard ecosystem alias `np`:

```
Python  
import numpy as np
```

```
# Verify successful setup and check version  
print(np.__version__)
```

2. Multi-Dimensional Arrays (`ndarray`) The core object in NumPy is the `ndarray` (n-dimensional array). It can scale across structural depths seamlessly, but must remain **homogeneous** (every single element inside the array must share the exact same data type and consistency).

```
Python  
# 0-D Array (A single scalar value point)  
arr_0d = np.array(42)  
  
# 1-D Array (A single flat row vector)  
arr_1d = np.array(['a', 'b', 'c'])
```

```
# 2-D Array (A matrix layout containing rows and columns)
arr_2d = np.array(['a', 'b', 'c'], ['d', 'e', 'f'])

# Checking the structural dimension count
print(arr_2d.ndim) # Outputs: 2(if 2D array then o/p will be 2)
```

Shape Consistency Rule

If your data has inhomogeneous shapes (e.g., one row has 3 elements but another row has only 2), NumPy will crash with a `ValueError`. Always insert clean placeholders (like a space or an underscore) to keep array boundaries structurally symmetrical.

Understanding Array Shapes (`.shape`)

The `.shape` attribute returns a tuple mapping out the layout boundaries of an array as (Depth/Layers, Rows, Columns).

```
Python
# A 3D layered array setup
arr_3d = np.array([['a', 'b'], ['c', 'd']],
                  [['e', 'f'], ['g', 'h']])

print(arr_3d.shape) # Outputs: (2, 2, 2) -> 2 layers, 2 rows, 2 columns
```

Multi-Dimensional Indexing

Avoid native Python chain indexing (`arr[0][0][0]`) because it creates unnecessary intermediate array objects. Instead, use comma-separated multi-dimensional indexing inside a single set of brackets.

```
Python
# Fast multi-dimensional lookup
element = arr_3d[0, 1, 0] # Layer 0, Row 1, Column 0
```

3. Advanced Slicing Techniques Slicing expressions allow you to carve out precise chunks from your data using the format: `[start : end : step]`. Remember, the `end` index parameter is always **exclusive**.

```
Python
# Setup a 4x4 sample matrix
matrix = np.array([
```